Chapter 12: Binary Search

What Problem Does Binary Search Solve?

Suppose:

```
1 3 5 7 9 11 13
```

Find:

```
11
```

Linear Search

```
for(int x : arr)
```

Complexity:

```
O(n)
```

Binary Search

Check middle.

```
1 3 5 7 9 11 13
      ^
```

Is 11 > 7 ?

Yes.

Ignore left half.

Now:

```
9 11 13
```

Check middle.

Found.

Complexity:

```
O(log n)
```

Very fast.

Important Condition

Array MUST be sorted.

```
1 3 5 7 9
```

✅

```
5 1 9 3 7
```

❌

Binary search won't work.

STL Function

Header:

```
#include <algorithm>
```

Syntax

```
binary_search(begin,end,value)
```

Example

```
vector<int> v={1,3,5,7,9};

cout<<binary_search(v.begin(),v.end(),7);
```

Output:

```
1
```

(True)

Example

```
cout<<binary_search(v.begin(),v.end(),10);
```

Output:

```
0
```

(False)

Return Type

Returns:

```
bool
```

Example

```
if(binary_search(v.begin(),v.end(),x))
{
```

```
    cout<<"Found";
}
else
{
    cout<<"Not Found";
}
```

Complexity

```
O(log n)
```

Array Example

```
int arr[]={1,3,5,7,9};

cout<<binary_search(arr,arr+5,5);
```

Output:

```
1
```

Common Interview Mistake

Wrong:

```
vector<int> v={5,1,7,3};

binary_search(v.begin(),v.end(),7);
```

❌ Undefined result

Because vector is not sorted.

Correct:

```
sort(v.begin(),v.end());

binary_search(v.begin(),v.end(),7);
```

Manual Binary Search

Interviewers may ask this.

```
int l=0;
int r=n-1;

while(l<=r)
{
    int mid=l+(r-l)/2;

    if(arr[mid]==x)
        return mid;

    else if(arr[mid]<x)
        l=mid+1;

    else
        r=mid-1;
}
```

STL vs Manual

STL:

```
binary_search()
```

Tells:

```
Exists?
```

Manual:

```
Binary Search
```

Can give:

```
Index
First Occurrence
Last Occurrence
Answer Space
```

More powerful.

Binary Search on Set

Works.

```
set<int> s={1,3,5,7};

cout<<binary_search(
    s.begin(),
    s.end(),
    5
);
```

But better:

```
s.find(5)
```

Complexity:

```
O(log n)
```

Binary Search on Vector of Pairs

Example:

```
vector<pair<int,int>> v=
{
    {1,2},
    {2,3},
    {4,5}
};
```

Search:

```
binary_search(
    v.begin(),
    v.end(),
    make_pair(2,3)
);
```

Output:

```
true
```

Binary Search vs Find

Vector:

```
find()
```

Complexity:

```
O(n)
```

Binary Search:

```
binary_search()
```

Complexity:

```
O(log n)
```

If sorted.

Real Interview Uses

---

1. Search in Sorted Array

LeetCode #704

---

2. First Bad Version

Binary Search on answer.

---

3. Search Insert Position

LeetCode #35

---

4. Square Root

Binary Search on answer.

---

5. Allocate Books

Binary Search on answer.

---

6. Aggressive Cows

Binary Search on answer.

Practice Questions

Easy

Q1

Input:

```
1 3 5 7 9
```

Search:

```
7
```

Output:

```
Found
```

Q2

Input:

```
1 3 5 7 9
```

Search:

```
8
```

Output:

```
Not Found
```

Q3

Sort array then search x.

Q4

Count how many queries exist.

Example:

```
Array:
1 2 3 4 5
```

```
Queries:
2 6 4
```

Output:

```
Yes
No
Yes
```

Medium

Q5

Find index using manual binary search.

Q6

Search in sorted vector of pairs.

Q7

Check if duplicate exists using sort + binary search.

Q8

Search string in sorted vector.

Interview Level

Q9

Search Insert Position

LeetCode #35

Q10

Search in Rotated Sorted Array

LeetCode #33

Q11

Find Peak Element

LeetCode #162

Q12

First Bad Version

LeetCode #278

Revision Sheet

Header

```
#include <algorithm>
```

Syntax

```
binary_search(begin,end,x);
```

Return

```
bool
```

Requirement

```
Sorted Data
```

Complexity

```
O(log n)
```

Example

```
vector<int> v={1,2,3,4,5};

binary_search(
    v.begin(),
    v.end(),
    4
);
```

Output:

```
true
```

Important Insight

`binary_search()` only answers:

```
Does x exist?
```

But interviews often ask:

```
First position?
Last position?
First >= x ?
First > x ?
```

For those we use:

```
lower_bound()
upper_bound()
```

which are even more important than `binary_search()` in real interviews.
